

AI workflow 導入現況

樂力專案

AI workflow 導入現況總覽

渠道對接 · 既有專案維運 · AI 客服機器人

2026 年 6 月

三大導入主軸

01

渠道對接相關專案

channels / ll-chnl / ll-chnl-reverse 構成「底層交互 → 正向對接 → 反向校正」的閉環,自動產出渠道規格與轉換模板。

02

樂力既有專案維運

以 multipay 系列為核心,讓 AI 由 Issue 接單,定位並增修既有龐大專案集的程式碼。

03

樂力 AI 客服機器人

解決查單、催單等客服場景;評估通用智能體框架與自建 Harness 兩條技術路線。

導入綜述

3

大導入主軸

渠道、維運、客服全面試行

7+

個專案

涵蓋既有 Java 與新建 AI 服務

AI

接單到產出

由 GitLab Issue 自動驅動工作流

核心做法

- 以 Claude Code 為主要 AI Agent,搭配專屬 Skills 標準化分析、開發與測試流程。
- 從 GitLab Issue 自動接單,讓 AI 完成需求分析、規格彙整、程式修改與模板產出。
- 強調「先完整分析、再產出索引文件」,協助 AI 在龐大既有專案中快速定位目標。
- 客服機器人並行評估通用框架與自建狀態機 Harness,以兼顧彈性與流程可控性。

01

PART ONE

渠道對接相關專案

channels、ll-chnl、ll-chnl-reverse 三者構成渠道對接的閉環生態。

三大專案定位

channels

渠道通用底層

核心基礎建設(既有 Java Spring 專案)。負責渠道通用的底層交互邏輯,提供穩定、標準化的傳輸與通訊基礎。

ll-chnl

正向渠道對接

業務對接與資料轉換核心。讀取需求並分析渠道文檔,對接接口,產出資料轉換模板與配置參數。

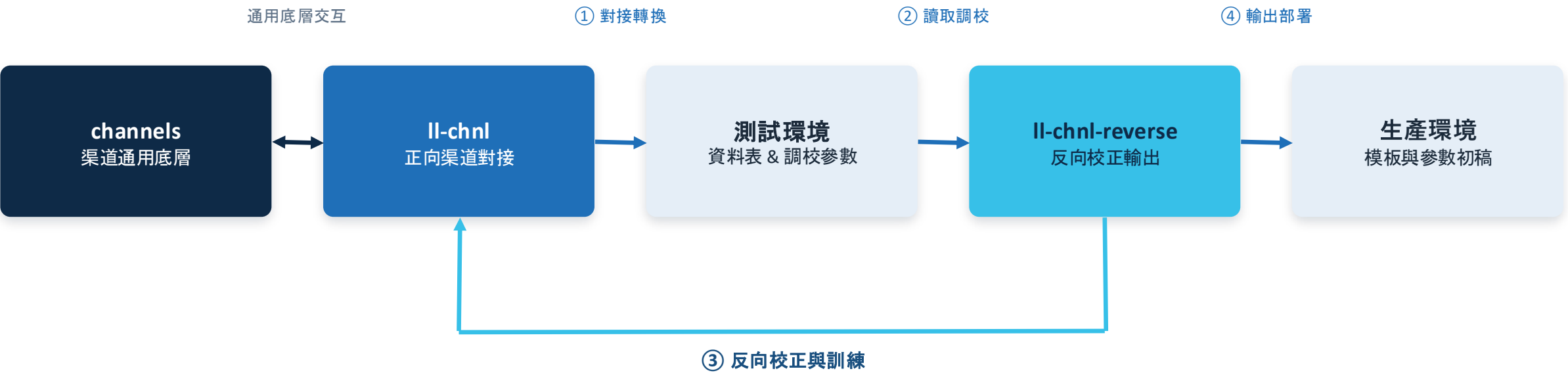
ll-chnl-reverse

反向校正與生產輸出

資料反向工程與品保工具。讀取測試環境調校後的資料,反向生成模板與 SQL,校正並訓練 ll-chnl,產出生產初稿。

閉環架構與資料流

「底層交互 → 正向對接 → 反向校正與生產輸出」的閉環生態



AI 工作流

1

讀取需求

由 GitLab Issue 讀取開發需求，並正規劃**需求規格**

2

彙整規格

分析本地/線上渠道文檔,產出統一格式**規格文檔**

3

產出模板

依 Skill 生成報文轉換模板與參數(含 SQL)

4

參考底層

視需要參考或修改 channels 程式碼

5

配置測試

將成果配置至測試環境

6

校正上線

OP 驗證後由 reverse 產出生產初稿

工具、挑戰與心得

使用工具

AI Agent

- Claude Code

Skills

- vibe-sdlc* (系列)
- analyze-channel-api
- analyze-channel-dev-requirement
- channel-dev
- git-workflow
- run-sql
- ll-kibana
- ast-grep
- agent-browser
- playwright-cli

主要挑戰

- 需整合跨專案邏輯、系統渠道規格、開發需求,以及外部渠道接口與規格。
- 渠道文檔常不清或不全、線上文檔繁雜,甚至需登入授權才能存取。

導入心得

- 從渠道文檔分析出發,產出正規化的渠道開發規格。
- 對接前先預先分析 channels,指示可參考的項目與修改重點。

02

PART TWO

樂力既有專案

以 multipay 系列為核心,讓 AI 接手龐大既有專案集的維運與優化。

專案定位與 AI 工作流

multipay 系列

樂力主要專案集

Java Spring / 既有專案。參考既有核心服務、後端與前端,依需求進行增修或優化。

使用工具

Claude Code · Antigravity IDE
vibe-sdlc* · git-workflow · ll-kibana
multipay-add-txn-type
design-doc-mermaid · spring-mvc-refactor-kit

- 1 由 AI 從 GitLab Issue 接單
- 2 告知修改重點與目標程式碼位置、參考文件，並**審查AI的執行計畫**
- 3 AI 依需求與提示修改程式碼
- 4 提交、MR、審查、合併並校驗
- 5 人工測試

主要挑戰與導入心得

主要挑戰

既有專案集龐大複雜,不易定位需修正的目標程式碼。

導入心得

先做一次完整分析

分析前列出所有相關專案與定位,讓 AI 依序分析,並為每個專案產出固定格式的分析文件。

生成索引文件

讓 AI 分析並建立索引,以快速定位目標文件與程式碼。

優化測試流程

期望自動佈署至測試環境後由 AI 進行測試,取代部分 OP 測試工作。

定義常用 Skill

針對常見修改建立專屬 skill,直接說明重點程式碼位置與修改重點。

03

PART THREE

樂力 AI 客服機器人

解決查單、催單等客服場景,並評估兩條技術路線。

專案目標與組成

專案目標 解決樂力運營的客服問題,提供「查單」、「催單」、「不明單查單」等功能,並兼具運營助理角色。

`agent-container`

AI Agent / TG Bot 佈署容器

以 OpenClaw 為基底,建構 TG Bot 插件與技能,佈署 Agents 並橋接 TG Bot。

`icpay-tg-bot-services`

AI Agent 業務服務

經由安全驗證機制提供必要的樂力業務服務,並包裝成 skill 供 Agent 使用。

`icpay-ai-bot-crm`

客服智能體框架 (Harness)

以預定義工作流(狀態機)控制對話;LLM 僅負責 NLU 與制式 Response,將意圖轉為事件。

兩種開發方向比較

評估方向:① 以通用智能體框架(OpenClaw / Hermes)為主 ② 自建 Harness 框架,嚴格控制業務流程

決策點	自建 Harness 框架 ✓ 受控	通用框架 (OpenClaw / Hermes)
主流程驅動	狀態機(transitions 庫)	LLM Agent 自主規劃下一步
LLM 輸出處理	翻譯層 + JSONLogic 規則	LLM 直接決定 action
工具暴露	自建 LLM 專屬 API 層(白名單)	將現有 API 直接餵給 Agent
業務規則	規則表 + 程式守衛	塞進 Prompt 讓 LLM 判斷
流程編排	狀態轉換表(顯式、可驗證)	Agent 的 reasoning chain

主要挑戰與發展心得

主要挑戰

客服業務邏輯複雜,且觸發詞相近;即使正確區分觸發詞意,後續流程仍變化多端,導致業務流程難以滿足運營需求。

通用智能體框架方向

需求應由簡至繁,循序迭代,逐步擴充能力。

自建 Harness 框架方向

定義強邏輯工作流(狀態機),確保每個流程步驟均可確定、可驗證。

業界佐證 · Claude Code 「Dynamic Workflow」

Anthropic 於 2026 年 5 月釋出的 Claude Code 「Dynamic Workflow」同樣將控制流程從 LLM 手中抽離、釘進確定性程式碼,LLM 僅承擔有邊界的工作——與自建 Harness 核心思路不謀而合,印證此為面對複雜業務流程、降低不確定性的主流選擇。

綜合心得與後續方向

- 面對龐大或複雜的既有專案,「先完整分析、再**產出索引**」有助快速定位程式碼、降低改錯風險,並留下可重用的結構地圖。
- 以 Skill 沉澱常見任務的重點與步驟,讓 AI 產出更穩定, 但也不要安裝太多同質或類似的 skills , 避免AI 混淆。
- 推進測試自動化, 讓 AI 接手部分 OP 測試步驟,縮短驗證循環。
- 樂力客服專案因流程確定性要求高,適合把「**控制流程**」從 LLM 抽離、釘進確定性程式碼(自建 Harness)以確保可控;但這並非唯一解——Claude Code「動態規劃 + 落地為確定性程式碼」的混合式思路同樣值得借鏡,宜依場景彈性選擇。
- 需求規格仍需以人為主導;AI 的價值在於協助釐清不明確的部分、補強遺漏,而非從零完整生成。
- AI 導入應循序漸進、逐步替換,而非一步到位。